

Social Network

Software Requirements Specification

27 March 2026

Ubeydullah Tuna – 150119713

Abdullah Puskar – 150119811

Muhammed Emin Erdoğan – 150123048

Yunus Kaya – 150119059

Prepared for
CSE3044 Software Engineering Term Project

Table of Contents

1. Introduction	2
1.1 Purpose	2
1.2 Scope	3
1.3 Definitions, Acronyms, and Abbreviations	3
1.4 References	3
1.5 Overview	4
2. General Description	4
2.1 Product Perspective	4
2.2 Product Functions	4
2.3 User Characteristics	4
2.4 General Constraints	4
2.5 Assumptions and Dependencies	5
3. Specific Requirements	5
3.1 External Interface Requirements	5
3.2 Functional Requirements	6
3.3 Non-Functional Requirements	11
3.4 Inverse Requirements	11
3.5 Design Constraints	12
3.6 Logical Database Requirements	12
4. UML Diagrams	13
4.1 Use Cases	13
4.2 Classes / Objects	14
5. Appendix	15
A.1: Decision Log	15
B.1: Class Diagram	17
B.2: Use Case Diagram	18
B.3: Sequence Diagram	19

1. Introduction

This Software Requirements Specification (SRS) document describes the requirements for the Social Network system. It provides a complete description of the system's intended behavior, constraints, and quality attributes to guide design, implementation, and testing.

1.1 Purpose

The purpose of this SRS is to define the functional and non-functional requirements for the Social Network system. The intended audience is the development team and the CSE3044 course instructor.

1.2 Scope

Social Network is a web-based social media platform with a lightweight HTML/CSS interface and a RESTful API. The system will:

- Allow users to register with interest selection, log in, and manage their profiles.
- Enable users to follow and unfollow other users.
- Allow users to create posts containing text and optional images, like posts, and comment on posts.
- Generate a personalized feed showing posts from followed users.
- Recommend content based on the user's selected interests and ongoing interaction history.
- Allow users to send and receive direct messages.
- Notify users about new followers, likes, and comments on their posts.

The system will not support real-time messaging (WebSocket), video uploads, story features, or third-party authentication.

1.3 Definitions, Acronyms, and Abbreviations

Term	Definition
API	Application Programming Interface – a set of rules enabling software components to communicate.
REST	Representational State Transfer – an architectural style for web APIs using HTTP methods.
JWT	JSON Web Token – a compact token used for stateless authentication.
SRS	Software Requirements Specification – this document.
Feed	A personalized list of posts from followed users, sorted by recency.
Hashtag	A keyword prefixed with # used to categorize post content.
Interest	A topic category selected by the user during registration (e.g., Technology, Sports).
Recommendation	A post suggested to the user based on their interests and interaction history.
DM	Direct Message – a private text message sent between two users.
Notification	An alert triggered when another user interacts with one's content.
ORM	Object-Relational Mapping – a technique to interact with a relational database using code objects.

1.4 References

- IEEE Std 830-1998, IEEE Recommended Practice for Software Requirements Specifications.
- CSE3044 Software Engineering Term Project Guidelines.

1.5 Overview

Section 2 provides a general description of the product, its context, users, and constraints. Section 3 contains detailed functional and non-functional requirements. Section 4 includes UML diagrams supporting the requirements.

2. General Description

2.1 Product Perspective

Social Network is a standalone, self-contained web application. It consists of a lightweight HTML/CSS frontend served by the same Node.js application that exposes the REST API. It does not depend on any external platform or third-party service.

2.2 Product Functions

The following is a summary of the major functions the system will perform:

- User Registration with Interest Selection – new users choose interest categories during sign-up.
- Authentication – users log in and receive a JWT token for subsequent requests.
- Profile Management – users can view and update their profile information.
- Follow / Unfollow – users subscribe to or unsubscribe from other users' content.
- Post Management – users create posts containing text and an optional image; posts may include hashtags.
- Likes and Comments – users can like posts and add or delete comments.
- Personalized Feed – reverse-chronological list of posts from followed users.
- Content Recommendation – posts suggested based on selected interests and interaction history (likes, comments).
- Direct Messaging – users can send and read private text messages to/from other users.
- Notifications – alerts for new followers, likes, and comments.

2.3 User Characteristics

- Unauthenticated User: can access only the registration and login pages.
- Registered User: authenticated user with full access to all features.

No domain-specific expertise is required. The interface is designed to be simple and self-explanatory for general internet users.

2.4 General Constraints

- The UI must be implemented in plain HTML/CSS, covering login, registration (with interest selection), feed, and post views.
- All server-side logic must be implemented in Node.js with Express.js.
- A relational database must be used for all persistent data storage.
- Authentication must be token-based using JWT. Passwords must never be stored in plaintext.
- Image uploads are stored on the server filesystem; external storage services are not required.

- Direct messaging is REST-based only; real-time delivery via WebSocket is out of scope.
- The project must be completed within a single academic semester.

2.5 Assumptions and Dependencies

- A Node.js-compatible server environment is available.
- A relational database is configured and accessible before the system starts.
- The server filesystem has sufficient space for uploaded image files.
- Clients store and send the JWT token in the Authorization header for protected requests.
- The interest-based recommendation uses a simple scoring rule: posts whose hashtags match the user's interest list or past interactions are ranked higher. No machine learning is applied.

3. Specific Requirements

3.1 External Interface Requirements

3.1.1 User Interfaces

The system provides a minimal HTML/CSS interface covering the following pages:

- Registration page – form for username, email, password, and interest selection (checkboxes).
- Login page – form for email and password.
- Feed page – displays posts from followed users and a recommendation section.
- Post detail page – shows a single post with its comments and like count.
- Profile page – shows user info, their posts, follower/following counts.
- Messages page – shows conversation list and a simple message thread view.

No JavaScript framework is required; the interface may use plain HTML forms and basic CSS styling.

3.1.2 Hardware Interfaces

No dedicated hardware interfaces are required. The system runs on any standard machine capable of hosting a Node.js application.

3.1.3 Software Interfaces

- Node.js runtime environment.
- Express.js framework for HTTP request handling and static file serving.
- A relational database management system (e.g., PostgreSQL).
- An ORM (e.g., Prisma) for schema management and queries.
- Multer (or equivalent) for handling multipart/form-data image uploads.

3.1.4 Communications Interfaces

The browser communicates with the server over HTTP. API endpoints accept and return JSON. Image upload endpoints accept multipart/form-data. Authentication uses JWT tokens in the Authorization header (Bearer scheme).

3.2 Functional Requirements

3.2.1 User Registration with Interest Selection

3.2.1.1 Introduction

New users create an account and select at least one interest category from a predefined list during registration.

3.2.1.2 Inputs

- username, email, password.
- Selected interest categories (e.g., Technology, Sports, Music, Art, Science, Travel).

3.2.1.3 Processing

- System validates uniqueness of username and email.
- System hashes the password.
- System stores the user record and the selected interest associations.

3.2.1.4 Outputs

- HTTP 201 with the created user ID; user is redirected to the login page.

3.2.1.5 Constraints

- Username: 3–30 characters, unique.
- Email: valid format, unique.
- Password: minimum 8 characters.
- At least one interest must be selected.

3.2.1.6 Error Handling

- Duplicate username or email → HTTP 409 Conflict.
- No interest selected → HTTP 400 Bad Request.

3.2.2 Authentication

3.2.2.1 Introduction

Registered users log in with their credentials and receive a JWT token.

3.2.2.2 Inputs

- Login: email, password.

3.2.2.3 Processing

- System verifies the provided credentials.
- System generates and returns a signed JWT token.
- Logout invalidates the token.

3.2.2.4 Outputs

- Login: HTTP 200 with JWT token; user is redirected to the feed page.
- Logout: HTTP 200 confirmation; user is redirected to the login page.

3.2.2.5 Constraints

- JWT tokens expire after a defined period.

3.2.2.6 Error Handling

- Invalid credentials → HTTP 401 Unauthorized.

3.2.3 Profile Management

3.2.3.1 Introduction

Users can view any profile and update their own display name, bio, and interest selections.

3.2.3.2 Inputs

- View: user ID or username in the request path.
- Update: optional fields – display name, bio, updated interest list.

3.2.3.3 Processing

- System retrieves the user record and their post count, follower count, and following count.
- For updates, only provided fields are modified; interest associations are replaced with the new selection.

3.2.3.4 Outputs

- HTTP 200 with the user profile object.

3.2.3.5 Constraints

- A user can only update their own profile.

3.2.3.6 Error Handling

- Nonexistent user → HTTP 404 Not Found.
- Unauthorized update → HTTP 403 Forbidden.

3.2.4 Follow / Unfollow

3.2.4.1 Introduction

Users can follow or unfollow other users.

3.2.4.2 Inputs

- Target user ID in the request path.

3.2.4.3 Processing

- Follow: a follow record is created and a notification is sent to the target user.
- Unfollow: the follow record is removed.

3.2.4.4 Outputs

- HTTP 200 confirmation.

3.2.4.5 Constraints

- A user cannot follow themselves.
- Duplicate follow requests are idempotent.

3.2.4.6 Error Handling

- Self-follow → HTTP 400 Bad Request.
- Target not found → HTTP 404 Not Found.

3.2.5 Post Management (Text + Image)

3.2.5.1 Introduction

Users can create posts containing text and an optional image, and delete their own posts.

3.2.5.2 Inputs

- Create: text content, optional image file (JPEG or PNG, max 5 MB).
- Delete: post ID in the request path.

3.2.5.3 Processing

- Create: system stores the post with author ID, timestamp, text content, image path (if provided), and extracted hashtags.
- Delete: system removes the post record, its associated image file, likes, and comments.

3.2.5.4 Outputs

- Create: HTTP 201 with the created post object including image URL if applicable.
- Delete: HTTP 200 confirmation.

3.2.5.5 Constraints

- Text content must not be empty. Maximum length is 500 characters.
- Accepted image formats: JPEG, PNG. Maximum file size: 5 MB.
- Only the author can delete their own post.

3.2.5.6 Error Handling

- Empty content or oversized/invalid image → HTTP 400 Bad Request.
- Unauthorized deletion → HTTP 403 Forbidden.

3.2.6 Likes and Comments

3.2.6.1 Introduction

Users can like or unlike posts and add or delete comments.

3.2.6.2 Inputs

- Like/Unlike: post ID.
- Comment: post ID, comment text (max 300 characters).
- Delete comment: comment ID.

3.2.6.3 Processing

- Like: a like record is created; a notification is sent to the post author.
- Unlike: the like record is removed.
- Comment: a comment record is created; a notification is sent to the post author; the post's hashtags are recorded as an interaction signal for recommendations.
- Delete comment: the comment record is removed.

3.2.6.4 Outputs

- HTTP 200/201 confirmation for each action.

3.2.6.5 Constraints

- A user can like a post only once.

- Only the author can delete their own comment.

3.2.6.6 Error Handling

- Duplicate like → HTTP 409 Conflict.
- Unauthorized comment deletion → HTTP 403 Forbidden.

3.2.7 Personalized Feed

3.2.7.1 Introduction

The system generates a reverse-chronological feed of posts from followed users.

3.2.7.2 Inputs

- Optional: page cursor for pagination.

3.2.7.3 Processing

- System retrieves the list of users the actor follows.
- Posts from those users are queried and sorted by creation date descending.
- Results are paginated with a maximum of 20 posts per page.

3.2.7.4 Outputs

- HTTP 200 with a paginated list of post objects (author info, image URL if present, like count, comment count).

3.2.7.5 Constraints

- Cursor-based pagination is used.

3.2.7.6 Error Handling

- If the user follows no one, an empty list is returned with HTTP 200.

3.2.8 Content Recommendation

3.2.8.1 Introduction

The system recommends posts from non-followed users based on the user's interest profile and interaction history.

3.2.8.2 Inputs

- No additional input; the system uses the actor's stored interests and interaction history.

3.2.8.3 Processing

- System collects the user's interest categories (set at registration, updatable via profile).
- System collects hashtags from posts the user has liked or commented on (interaction signals).
- System queries recent posts from non-followed users whose hashtags match any collected interest or interaction signal.
- Posts are ranked by the number of matching signals (interest matches + interaction matches).
- Top results are returned, excluding the user's own posts.

3.2.8.4 Outputs

- HTTP 200 with a list of up to 10 recommended post objects.

3.2.8.5 Constraints

- Recommendation is rule-based scoring only; no machine learning is used.
- The user's own posts are never recommended.

3.2.8.6 Error Handling

- If no signals exist, the most recently created popular posts are returned as a fallback.

3.2.9 Direct Messaging

3.2.9.1 Introduction

Users can send and read private text messages to and from other registered users.

3.2.9.2 Inputs

- Send: recipient user ID, message text (max 1000 characters).
- Read: conversation partner's user ID.

3.2.9.3 Processing

- Send: system stores the message with sender ID, recipient ID, text, and timestamp.
- Read: system retrieves all messages between the two users, sorted by timestamp ascending.

3.2.9.4 Outputs

- Send: HTTP 201 with the created message object.
- Read: HTTP 200 with a list of message objects.

3.2.9.5 Constraints

- Message delivery is not real-time; the recipient sees new messages upon refreshing the page.
- Only participants of a conversation can read its messages.

3.2.9.6 Error Handling

- Recipient not found → HTTP 404 Not Found.
- Empty message text → HTTP 400 Bad Request.
- Unauthorized conversation access → HTTP 403 Forbidden.

3.2.10 Notifications

3.2.10.1 Introduction

Users receive notifications about interactions with their content.

3.2.10.2 Inputs

- View: no input required.
- Mark as read: notification ID or 'all'.

3.2.10.3 Processing

- Notifications are created automatically on follow, like, and comment events.
- User retrieves all notifications sorted by recency with is_read status.
- User can mark one or all notifications as read.

3.2.10.4 Outputs

- HTTP 200 with a list of notification objects (type, actor username, is_read, created_at).

3.2.10.5 Constraints

- Notification types: new_follower, post_liked, post_commented.

3.2.10.6 Error Handling

- Invalid notification ID → HTTP 404 Not Found.

3.3 Non-Functional Requirements

3.3.1 Performance

- 95% of API requests shall be responded to within 500 milliseconds under normal load.
- Feed and recommendation endpoints shall return paginated results of up to 20 posts per request.
- The system shall support at least 50 concurrent users without performance degradation.
- Image upload processing shall complete within 3 seconds for files up to 5 MB.

3.3.2 Reliability

- The system shall validate all incoming data and return descriptive error messages for invalid input.
- Concurrent write operations shall be handled atomically to prevent data inconsistencies.
- Uploaded image files shall be stored persistently and remain accessible after server restart.

3.3.3 Availability

- The system shall remain available throughout evaluation and demonstration periods.
- Server-side errors shall be returned as HTTP 500 responses rather than causing process crashes.

3.3.4 Security

- All protected endpoints shall require a valid JWT token in the Authorization header.
- Passwords shall be hashed using bcrypt before storage.
- Users shall not be able to modify, delete, or access resources belonging to other users.
- All database queries shall use parameterized statements to prevent SQL injection.
- Uploaded files shall be validated for type and size before storage to prevent malicious uploads.

3.3.5 Maintainability

- The codebase shall be organized into distinct modules: routes, controllers, services, and models.
- Database schema shall be managed through migration files.
- All API endpoints shall be documented in a Postman collection or Swagger file.

3.3.6 Portability

- The system shall run on any OS that supports Node.js (Windows, macOS, Linux).
- All environment-specific configurations shall be managed via environment variables.

3.4 Inverse Requirements

- The system shall not support real-time messaging (no WebSocket implementation).
- The system shall not support video file uploads.

- The system shall not implement story or ephemeral content features.
- The system shall not use machine learning for content recommendation.
- The system shall not support third-party (OAuth) authentication.
- The system shall not send email notifications.

3.5 Design Constraints

- Server-side logic must be implemented in Node.js with the Express.js framework.
- A relational database must be used for all persistent data storage.
- Authentication must rely exclusively on JWT tokens.
- The frontend must be implemented in plain HTML/CSS without a JavaScript framework.
- Image files must be stored on the server filesystem in a designated uploads directory.

3.6 Logical Database Requirements

The following entities and their relationships are required:

Entity	Key Attributes	Relationships
users	id, username, email, password hash, bio, created_at	Has many posts, follows, messages, notifications
interests	id, name	Many-to-many with users via user_interests
user_interests	user_id, interest_id	Join table: users ↔ interests
posts	id, user_id, content, image_path, created_at	Belongs to user; has many likes, comments, hashtags
follows	follower_id, following_id, created_at	Join table between users
likes	id, user_id, post_id, created_at	Belongs to user and post
comments	id, user_id, post_id, content, created_at	Belongs to user and post
hashtags	id, name	Many-to-many with posts via post_hashtags
messages	id, sender_id, recipient_id, content, created_at	Belongs to sender and recipient (both users)
notifications	id, user_id, actor_id, type, post_id, is_read, created_at	Belongs to user; references actor

- Indexes shall be defined on foreign key columns and on columns used in ORDER BY and WHERE clauses.
- Cascading deletes shall apply: deleting a post removes its likes, comments, and image file.
- Data shall be retained indefinitely unless explicitly deleted by the owning user.

4. UML Diagrams

4.1 Use Cases

4.1.1 Use Case: Register with Interest Selection

Use Case ID	UC001
Use Case Name	Register with Interest Selection
Actors	Unauthenticated User
Description	A new user creates an account and selects at least one interest category.
Preconditions	The username and email must not already be in use.
Basic Flow	1. User fills in the registration form (username, email, password, interests). 2. System validates inputs and uniqueness. 3. System hashes password, stores user and interest associations. 4. User is redirected to the login page.
Post-condition	A new user record with interest associations exists. The user may log in.
Exceptions	Duplicate email/username → HTTP 409. No interest selected → HTTP 400.

4.1.2 Use Case: Content Recommendation

Use Case ID	UC008
Use Case Name	Content Recommendation
Actors	Registered User
Description	The system surfaces relevant posts from non-followed users based on the user's interests and interaction history.
Preconditions	The actor must be authenticated.
Basic Flow	1. User opens the feed page; recommendations section loads automatically. 2. System collects the user's interest categories and hashtags from liked/commented posts. 3. System scores non-followed users' posts by matching hashtags. 4. Top-scored posts are returned and displayed in the recommendations section.
Post-condition	User sees a list of relevant posts they may not have discovered otherwise.
Exceptions	No signals exist → most recent popular posts shown as fallback.

4.1.3 Use Case: Direct Messaging

Use Case ID	UC009
Use Case Name	Direct Messaging
Actors	Registered User
Description	A user sends a private text message to another registered user and reads their conversation.
Preconditions	Both users must be registered. The actor must be authenticated.
Basic Flow	1. User navigates to the Messages page. 2. User selects a recipient and types a message. 3. System stores the message and displays it in the conversation thread. 4. Recipient sees the message upon next page load.
Post-condition	A message record is stored. Both participants can read the conversation.
Exceptions	Recipient not found → HTTP 404. Empty message → HTTP 400.

4.2 Classes / Objects

4.2.1 User

4.2.1.1 Attributes

- id: integer, primary key
- username: string, unique, not null
- email: string, unique, not null
- password_hash: string, not null
- bio: string, nullable
- created_at: timestamp

4.2.1.2 Functions

- register(username, email, password, interests) – creates account [UC001]
- login(email, password) – authenticates and returns JWT [UC002]
- updateProfile(fields) – updates bio, display name, or interests [UC003]
- follow(targetUser) / unfollow(targetUser) – manages follow relationship [UC004]

4.2.2 Post

4.2.2.1 Attributes

- id: integer, primary key
- user_id: integer, foreign key → User
- content: string, not null, max 500 chars
- image_path: string, nullable

- created_at: timestamp

4.2.2.2 Functions

- create(content, imageFile) – creates post with optional image and hashtags [UC005]
- delete() – removes post, image file, likes, and comments [UC006]
- like() / unlike() – toggles like and triggers notification [UC007]
- addComment(text) / deleteComment(id) – manages comments [UC007]

4.2.3 Message

4.2.3.1 Attributes

- id: integer, primary key
- sender_id: integer, foreign key → User
- recipient_id: integer, foreign key → User
- content: string, not null, max 1000 chars
- created_at: timestamp

4.2.3.2 Functions

- send(recipientId, content) – stores a new message [UC009]
- getConversation(userId) – retrieves all messages between two users [UC009]

4.2.4 Recommendation Engine

4.2.4.1 Attributes

- userId: integer – the actor requesting recommendations
- interestTags: list of strings – from user's selected interests
- interactionTags: list of strings – hashtags from liked and commented posts

4.2.4.2 Functions

- collectSignals(userId) – merges interest tags and interaction tags
- scorePost(post) – counts how many collected signals match the post's hashtags
- recommend(userId) – returns top-scored posts from non-followed users [UC008]

Appendix A.1: Decision Log

This log documents the key engineering decisions made during the Requirements phase of the Social Network project. Each entry addresses the rationale, alternatives, and associated risks, as required by the CSE3044 project guidelines .

1. Technology Stack: Node.js and Express.js

- **Decision Made:** Utilize Node.js with the Express.js framework for all server-side logic.
- **Alternatives Considered:** Python (Django/Flask) or Java (Spring Boot).
- **Rationale:** This is a mandatory constraint defined in the project guidelines. Additionally, Node.js allows for a unified JavaScript stack, which accelerates development speed for a single-semester project.

- **Trade-offs:** Node.js is highly efficient for the I/O-bound nature of social media (feed retrieval, likes) but can be less performant for heavy CPU-bound tasks.
- **Risks / Technical Debt:** Requires careful management of asynchronous operations to ensure robust error handling.

2. Data Management: PostgreSQL and Prisma ORM

- **Decision Made:** Use a relational database (PostgreSQL) interfaced through the Prisma ORM.
- **Alternatives Considered:** NoSQL (MongoDB) or writing raw SQL queries.
- **Rationale:** Social network data such as followers, likes, and comments are inherently relational, making a RDBMS ideal for maintaining ACID compliance. Prisma provides type-safety and simplified schema migration management.
- **Trade-offs:** Prisma adds an abstraction layer that may slightly impact performance for extremely complex joins compared to hand-optimized raw SQL.
- **Risks / Technical Debt:** Frequent schema changes require disciplined migration tracking.

3. Authentication: JSON Web Tokens (JWT)

- **Decision Made:** Implement stateless authentication using JWT tokens passed in the Authorization header.
- **Alternatives Considered:** Session-based authentication with server-side session storage.
- **Rationale:** JWT supports a stateless RESTful architecture, allowing the system to scale without server-side session tracking. This is a standard requirement for the project.
- **Trade-offs:** JWTs cannot be easily revoked before they expire.
- **Risks / Technical Debt:** Security depends heavily on the client properly storing and securing the token.

4. Recommendation Logic: Rule-based Scoring

- **Decision Made:** Implement a recommendation engine using a simple scoring rule based on interest matching and interaction hashtags.
- **Alternatives Considered:** Machine Learning (ML) based recommendation models.
- **Rationale:** Project constraints explicitly prohibit the use of machine learning. A rule-based approach is transparent, testable, and fits the academic scope.
- **Trade-offs:** The system provides less sophisticated personalization than an ML model might offer.
- **Risks / Technical Debt:** The scoring algorithm may need optimization as the volume of hashtags and interactions grows .

5. User Interface: Plain HTML and CSS

- **Decision Made:** Develop the frontend using only plain HTML and CSS, served by the Node.js application.
- **Alternatives Considered:** Modern frontend frameworks like React, Vue, or Angular.
- **Rationale:** This is a mandatory design constraint to focus on core software engineering and server-side fundamentals.
- **Trade-offs:** Lacks component-based modularity, leading to more manual DOM management and potential code repetition.
- **Risks / Technical Debt:** Maintaining UI consistency across pages becomes more labor-intensive without a component framework.

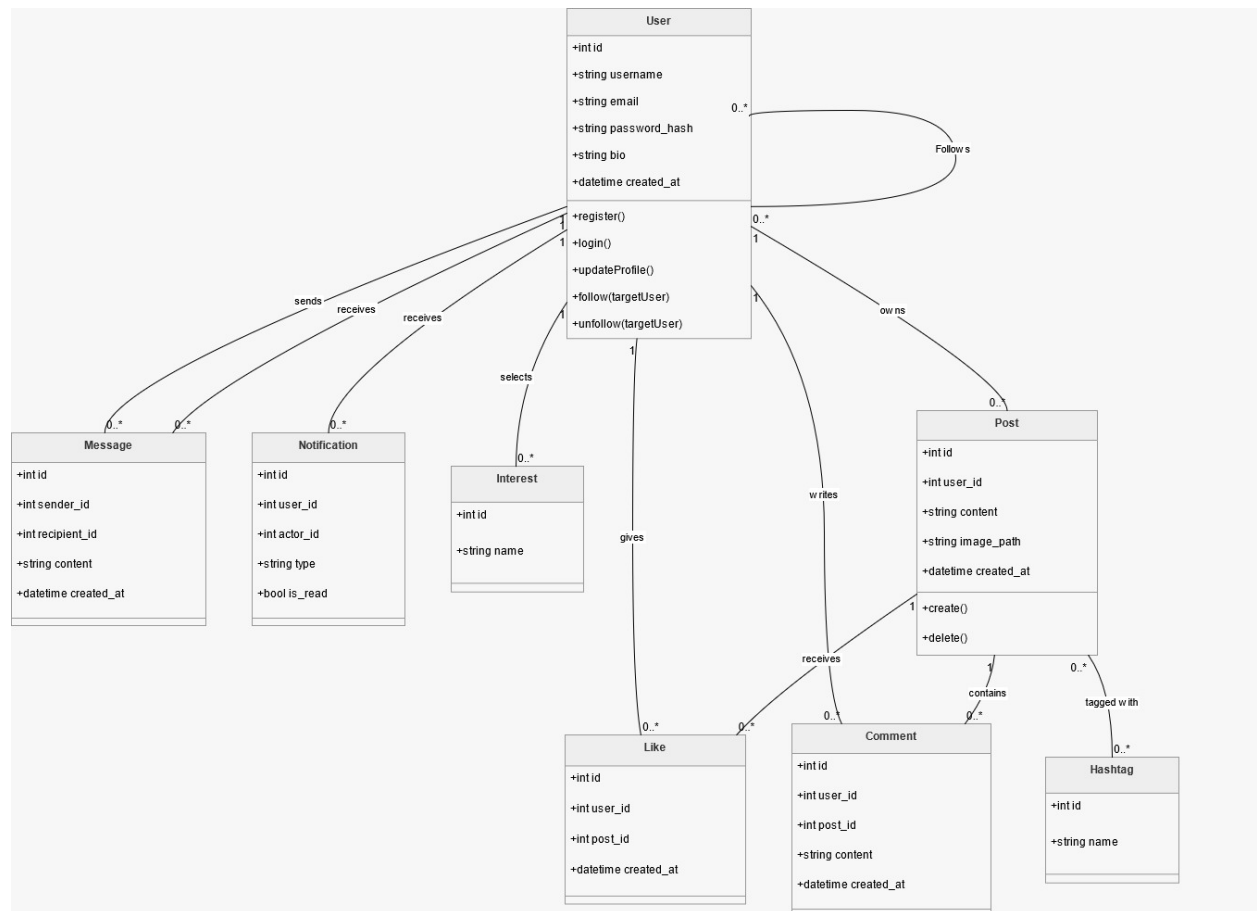
6. Media Storage: Local Filesystem

- **Decision Made:** Store uploaded user images on the server's local filesystem in a dedicated directory.
- **Alternatives Considered:** External cloud storage services (e.g., AWS S3).
- **Rationale:** Local storage simplifies infrastructure setup and avoids dependency on external paid services or complex API integrations for a prototype.
- **Trade-offs:** Makes horizontal scaling difficult as images are tied to a specific server instance.
- **Risks / Technical Debt:** Risk of media loss if the server instance is not backed up or is reset.

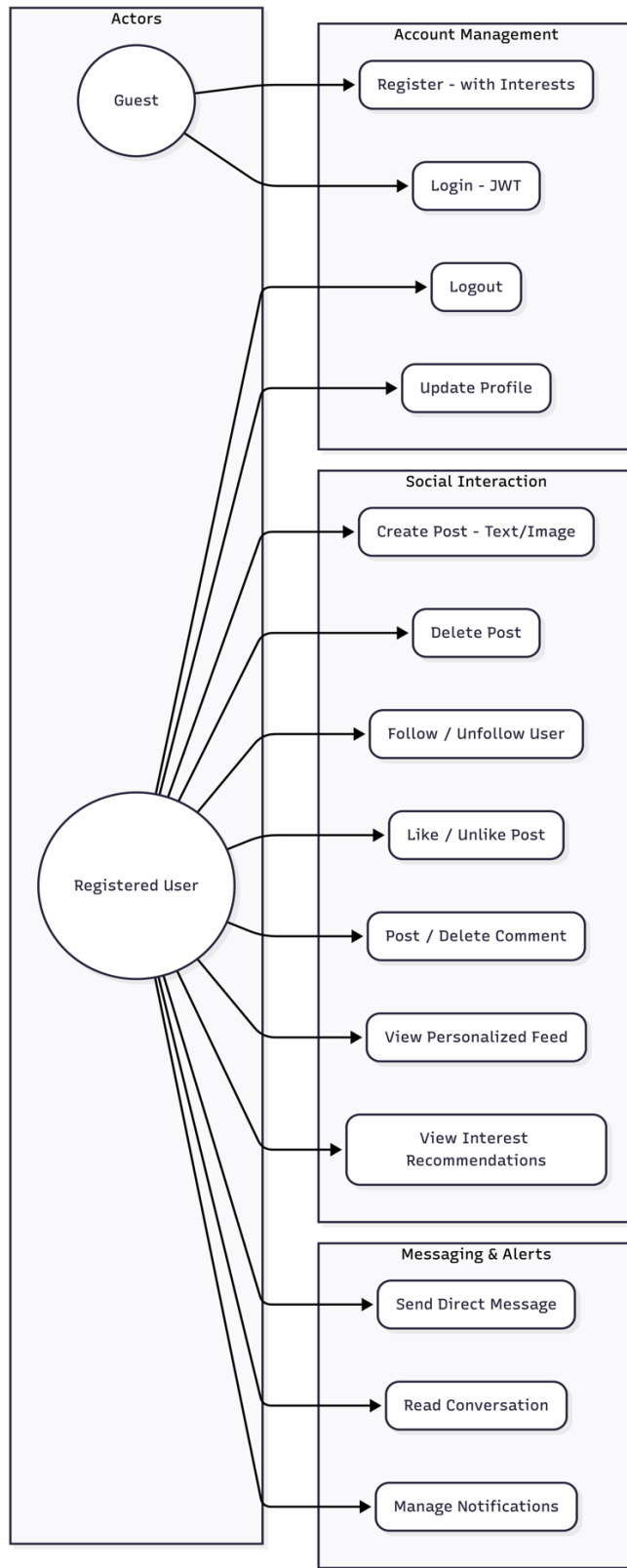
7. Communication: REST-based Messaging

- **Decision Made:** Implement direct messaging using standard RESTful endpoints without real-time implementation.
- **Alternatives Considered:** Real-time communication using WebSockets (Socket.io).
- **Rationale:** WebSockets are explicitly out of scope according to the project guideline. Users will see new messages upon page refresh.
- **Trade-offs:** The user experience is not real-time, which may feel dated compared to modern platforms.
- **Risks / Technical Debt:** Potential future demand for real-time features would require a significant architectural shift.

Appendix B.1: Class Diagram



Appendix B.2: Use Case Diagram



Appendix B.3: Sequence Diagram

